GUIA DE LABORATÓRIO

JAVA + POO + MODELAGEM + BANCO DE DADOS

Da classe e do objeto à entidade LIVRO, SQL, JDBC e interface Swing.

NA PRÁTICA | Material de estudo, revisão e prática. Indicado também para quem perdeu a aula.

FACULDADE | TRILHA DIDÁTICA • 2026

Como usar este guia

O objetivo não é decorar comandos: é entender como as partes de um sistema se conectam. Leia as analogias, execute os exemplos e depois resolva os desafios. Os códigos completos e o script SQL acompanham este PDF em um projeto ZIP; há também uma página HTML interativa para revisão.

01 | Fundamentos — classe, objeto, atributo, método e construtor

02 | Celular — entendendo e melhorando o código da aula

03 | Modelagem — os 100 atributos de LIVRO e suas entidades corretas

04 | Banco de dados — tabelas, PK, FK, tipos e SQL

05 | JDBC — como Java conversa com MySQL

06 | Swing — JFrame, JTextField, layout e ação do botão

07 | Missões — erros comuns, exercícios e respostas

NA PRÁTICA | Se você faltou: siga na ordem, mesmo que já saiba um pouco de Java. Os capítulos são conectados.

01 | O mundo real vira código

Classe, objeto, atributo e método

Pense em uma planta de uma casa: a planta define o que a casa deve ter. A casa construída é uma instância concreta dessa planta. Em Java, a planta é a CLASSE e a casa é o OBJETO. Os atributos descrevem o objeto; os métodos dizem o que ele faz.

Exemplo: classe DadosCelular; objeto celular01; atributos modeloCelular, precoCelular e usado; método exibirDadosCelular().

```
DadosCelular celular01 = new DadosCelular();
celular01.modeloCelular = "Galaxy S24";
celular01.usado = false;
celular01.exibirDadosCelular();
```

Decodificando linha a linha

DadosCelular = tipo/classe; celular01 = variável de referência; new = cria o objeto; DadosCelular() = chama o construtor; o ponto (.) acessa um atributo ou um método. Vários celulares podem nascer da mesma classe e ter valores diferentes.

public, private e encapsulamento

No exemplo original, os campos não têm modificador: têm acesso de pacote, não são public. A classe ExibirDadosCelular consegue usá-los porque está no mesmo pacote. Em sistemas maiores, prefira atributos private com construtor e métodos de acesso ou comportamentos específicos.

```
public class Celular {
    private String modelo;
    public Celular(String modelo) { this.modelo = modelo; }
    public String getModelo() { return modelo; }
}
```

NA PRÁTICA | Encapsular é como ter um caixa eletrônico: você usa uma interface controlada, sem alterar diretamente o dinheiro lá dentro.

02 | Seu exemplo: DadosCelular

O código original da turma usa duas classes: DadosCelular (estrutura e comportamento) e ExibirDadosCelular (o método main, que inicia a execução). O pacote POO.celular agrupa essas classes. Em projetos profissionais é mais comum escrever nomes de pacote em minúsculas, como poo.celular.

O fluxo que realmente acontece

1. main começa. 2. new DadosCelular() cria o celular. 3. A aplicação preenche campos. 4. exibirDadosCelular() imprime os valores. 5. printf substitui cada %s ou %.2f pelo valor correspondente, na mesma ordem. %n faz uma quebra de linha portátil.

```
System.out.printf("Modelo: %s | Preco: R$ %.2f%n",
        modeloCelular, precoCelular);
System.out.println(estadoCelular ? "Usado" : "Novo");
```

Operador ternário e boolean

A expressão condição ? resultadoSeVerdadeiro : resultadoSeFalso é uma decisão compacta. Na sua classe: false significa Novo; true significa Usado. Seria mais claro renomear estadoCelular para usado, indicando o sentido do boolean.

Importações e tipos

O import java.sql.Blob da primeira classe importa um tipo que representa dados binários grandes do JDBC. Ele aparece no atributo imageCelular, mas a imagem não é usada na impressão e não é necessária para aprender POO. Se quiser guardar imagens, há também a alternativa de armazenar o caminho/URL em vez dos bytes.

float funciona no exercício, mas para valores monetários reais prefira BigDecimal, porque tipos binários de ponto flutuante podem representar decimais de forma aproximada.

```
import java.math.BigDecimal;
BigDecimal preco = new BigDecimal("4999.90");
System.out.println("R$ " + preco);
```

NA PRÁTICA | Detalhe importante: o nome de um processador varia conforme o modelo e a região do aparelho. Os valores do exemplo são ilustrativos e não uma ficha técnica verificada.

03 | De 100 atributos a um MER inteligente

As capturas mostram um exercício de modelagem de dados: levantar até 100 atributos possíveis relacionados à entidade LIVRO para depois classificá-los e desenhá-los no brModelo. A coleta inicial é propositalmente ampla. Na modelagem final, porém, nem tudo deve ficar dentro de uma única entidade.

Os oito grupos das capturas

Identificação (1–10) — ID, código interno, ISBN-10, ISBN-13, título, subtítulo, títulos alternativos, código de barras, DOI.

Obra (11–25) — sinopse, gênero, palavras-chave, idioma original, público-alvo, coleção e volume.

Autoria (26–40) — autor, coautor, tradutor, ilustrador e outros colaboradores.

Publicação (41–55) — editora, selo, edição, publicação, tiragem, licença e direitos.

Físico (56–70) — páginas, dimensões, encadernação, tipo de papel e ilustrações.

Digital (71–80) — formato, tamanho de arquivo, URLs, DRM, acessibilidade e data digital.

Comercial (81–90) — preço, promoções, desconto, custos, moeda e disponibilidade.

Biblioteca (91–100) — tombo, exemplar, localização, conservação, aquisição e situação.

O erro que muitos cometem

Jogar os 100 atributos numa única tabela LIVRO parece prático, mas causa repetição, valores vazios e atualizações inconsistentes. Autor tem dados próprios; um livro pode ter vários autores. Exemplar é a cópia física individual e não a obra. Promoções e preços podem mudar ao longo do tempo.

Uma separação mais coerente

OBRA → id_obra, titulo, sinopse, idioma_original

EDICAO → id_edicao, id_obra, isbn13, ano, numero_edicao, id_editora

AUTOR → id_autor, nome, nacionalidade

OBRA_AUTOR → id_obra, id_autor, papel

EDITORA → id_editora, nome

EXEMPLAR → id_exemplar, id_edicao, tombo, localizacao, estado

ARQUIVO_DIGITAL → id_arquivo, id_edicao, formato, url, drm

PRECO → id_preco, id_edicao, valor, inicio_vigencia

NA PRÁTICA | Obra ≠ edição ≠ exemplar. Uma obra pode ter várias edições; uma edição pode ter vários exemplares. ISBN tende a identificar a edição/formato, e tombo identifica cada cópia física.

04 | MER e cardinalidade sem mistério

MER significa Modelo Entidade-Relacionamento. Entidade é algo sobre o qual guardamos dados. Atributo descreve uma entidade. Relacionamento descreve como entidades se associam. Cardinalidade explica quantos registros podem participar dessa associação.

```
OBRA (1) -------- (N) EDICAO
EDICAO (1) ------ (N) EXEMPLAR
OBRA (N) ------- (N) AUTOR
                  | resolvido com OBRA_AUTOR |
EDITORA (1) ----- (N) EDICAO
```

Como montar no brModelo

1. Crie entidades e seus atributos. 2. Marque identificadores (chaves). 3. Desenhe relacionamentos e cardinalidades. 4. Confira se a multiplicidade é correta no caso real. 5. Resolva N:N com uma entidade/tabela associativa quando passar ao modelo lógico. 6. Revise atributos repetidos e atributos que dependem de outras entidades.

Exemplo real: uma biblioteca possui cinco exemplares da mesma edição. O título e o ISBN são dados da edição, mas número de tombo e estado de conservação são específicos de cada exemplar. Não repita título e ISBN cinco vezes sem necessidade.

Atributos especiais

Identificador: valor que distingue cada ocorrência. Composto: pode ser dividido (por exemplo, endereço). Multivalorado: admite vários valores (por exemplo, idiomas disponíveis); normalmente vira associação ou tabela relacionada. Derivado: calculado com outros dados (por exemplo, preço final). Opcional: pode ficar ausente.

05 | Do desenho ao SQL

Banco de dados é como um arquivo organizado. Tabela é um conjunto de registros do mesmo tipo; coluna é uma característica; linha é um registro. MySQL é um sistema gerenciador de banco de dados. SQL é a linguagem usada para criar e consultar esses dados.

PK, FK e restrições

PRIMARY KEY (PK): identifica de modo único uma linha. FOREIGN KEY (FK): liga uma tabela a outra. NOT NULL: valor obrigatório. UNIQUE: evita duplicatas no campo restrito. AUTO_INCREMENT: gera identificadores sequenciais no MySQL.

```
CREATE DATABASE IF NOT EXISTS biblioteca_aula
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE biblioteca_aula;

CREATE TABLE IF NOT EXISTS livro (
  id INT PRIMARY KEY AUTO_INCREMENT,
  titulo VARCHAR(200) NOT NULL,
  isbn13 CHAR(13) UNIQUE,
  ano_publicacao SMALLINT
);

-- Outros atributos do enunciado pertencem a entidades relacionadas:
-- autor, editora, exemplar, emprestimo, preco/historico_preco etc.
-- O codigo Java demonstra apenas a entidade basica LIVRO.
```

CRUD: as quatro operações

```
-- Create (inserir)
INSERT INTO livro (titulo, isbn13, ano_publicacao)
VALUES ('Algoritmos', '9780000000002', 2026);
-- Read (consultar)
SELECT id, titulo FROM livro;
-- Update (alterar)
UPDATE livro SET titulo = 'Algoritmos e Java' WHERE id = 1;
-- Delete (excluir)
DELETE FROM livro WHERE id = 1;
```

NA PRÁTICA | Nunca execute UPDATE ou DELETE sem entender o WHERE: sem filtro, você pode alterar ou excluir todos os registros.

06 | JDBC: a ponte entre Java e MySQL

JDBC (Java Database Connectivity) é a API Java para conversar com bancos de dados relacionais. O driver Connector/J interpreta as chamadas JDBC para o MySQL. Analogia: o Java é o cliente, JDBC é o balcão de atendimento, o driver é o intérprete e o banco é o arquivo que processa o pedido.

```
Aplicacao Java -> JDBC -> Driver MySQL -> Banco MySQL
                     <- resultados <-
```

Passo 1: conectar

```
package biblioteca;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexao {
    private static final String URL = "jdbc:mysql://localhost:3306/biblioteca_aula";
    private static final String USUARIO = System.getenv().getOrDefault("DB_USER", "root");
    private static final String SENHA = System.getenv().getOrDefault("DB_PASSWORD", "");

    public static Connection abrir() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, SENHA);
    }
}
```

Uma URL JDBC indica o protocolo, o servidor, a porta e o nome do banco. O MySQL costuma usar a porta 3306, mas isso é configurável. Verifique se o serviço está iniciado, se o schema foi criado e se o usuário tem permissão.

Passo 2: enviar SQL com segurança

```
String sql = "INSERT INTO livro (titulo, isbn13, ano_publicacao) " +
             "VALUES (?, ?, ?)";
try (Connection con = Conexao.abrir();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, "Introducao ao Java");
    ps.setString(2, "9780000000003");
    ps.setInt(3, 2026);
    ps.executeUpdate();
}
```

Os ? são parâmetros. PreparedStatement separa o comando SQL dos valores e é a escolha preferencial para consultas parametrizadas, reduzindo o risco de injeção SQL. executeUpdate() retorna contagem para INSERT/UPDATE/DELETE; executeQuery() é usado em SELECT e devolve um ResultSet.

07 | Ler os resultados do banco

```
String sql = "SELECT id, titulo FROM livro ORDER BY id";
try (Connection con = Conexao.abrir();
     PreparedStatement ps = con.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        int id = rs.getInt("id");
        String titulo = rs.getString("titulo");
        System.out.println(id + " - " + titulo);
    }
}
```

rs.next() move para a próxima linha. Enquanto houver linhas, o laço continua. getInt e getString leem o tipo esperado da coluna. O try-with-resources fecha Connection, PreparedStatement e ResultSet mesmo se surgir uma exceção.

Por que criar um DAO?

DAO (Data Access Object) separa a responsabilidade de acesso ao banco do restante da aplicação. A interface gráfica não precisa saber escrever SELECT. O DAO é a camada que executa as operações e transforma linhas em objetos Java.

```
Tela / main -> LivroDAO -> JDBC -> MySQL
                         <- List<Livro> <-
```

No ZIP você encontra Livro.java (objeto), Conexao.java (abertura), LivroDAO.java (inserção/listagem) e AppBiblioteca.java (teste). Estude o fluxo nesta sequência.

Riscos e boas práticas

Evite senha fixa no repositório; use variáveis de ambiente ou configuração protegida. Não concatene entradas de usuário no SQL. Feche os recursos. Valide dados antes de persistir. Na segunda execução do projeto de exemplo, o mesmo ISBN fere a regra UNIQUE; isso é intencionalmente explicado no README.

08 | Swing: construindo uma tela de cadastro

A última imagem mostra código com JFrame, JTextField, JPanel, GridLayout e setTitle("Cadastro de Livros"). É Java Swing, biblioteca de interface gráfica desktop. Ela é independente da conexão com banco: primeiro montamos a tela, depois ligamos os eventos ao DAO.

As peças da tela

JFrame é a janela. JPanel agrupa componentes. JTextField recebe texto. JLabel mostra rótulos. JButton dispara uma ação. GridLayout coloca itens em linhas e colunas. setSize define dimensões; setLocationRelativeTo(null) centraliza a janela; setDefaultCloseOperation define o fechamento.

```
JFrame janela = new JFrame("Cadastro de Livros");
JPanel painel = new JPanel(new GridLayout(3, 2, 10, 10));
JTextField campoTitulo = new JTextField();
JButton botaoSalvar = new JButton("Salvar");
painel.add(new JLabel("Titulo:"));
painel.add(campoTitulo);
painel.add(new JLabel("Acao:"));
painel.add(botaoSalvar);
janela.add(painel);
janela.setSize(450, 220);
janela.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
janela.setLocationRelativeTo(null);
janela.setVisible(true);
```

Como salvar de verdade?

O botão precisa de ActionListener. Dentro do evento, valide o campo e chame o método inserir do LivroDAO. Para aplicativos reais, não faça operações demoradas de banco na Event Dispatch Thread (EDT), pois a janela pode congelar; use SwingWorker ou tarefa de fundo.

```
botaoSalvar.addActionListener(e -> {
    String titulo = campoTitulo.getText().trim();
    if (titulo.isEmpty()) {
        JOptionPane.showMessageDialog(janela, "Informe um titulo.");
        return;
    }
    // Proximo passo: validar os outros campos
    // e chamar new LivroDAO().inserir(...)
});
```

09 | Mapa de erros que você vai encontrar

cannot find symbol → Nome de variável, método ou classe incorreto; confira letras e importações.

package ... does not exist → Caminho de pasta, package ou classpath incorreto.

No suitable driver found → Driver JDBC ausente do classpath em tempo de execução ou URL inadequada.

Communications link failure → MySQL fora do ar, endereço/porta errados ou rede inacessível.

Access denied for user → Usuário, senha ou permissões incorretos.

Unknown database → O banco especificado não existe; execute o script de criação.

Duplicate entry → Valor violou UNIQUE/PRIMARY KEY; no exemplo, ISBN repetido.

NullPointerException → Você tentou usar uma referência que está null; investigue a inicialização.

SQLSyntaxErrorException → Comando SQL malformado ou referência a tabela/coluna inexistente.

NA PRÁTICA | Estratégia de debug: leia o PRIMEIRO erro significativo, encontre a linha, valide nomes e tipos, teste o SQL isolado e então execute Java novamente.

10 | Missões de laboratório

Nível 1: POO — Crie celular02 com outras características. Exiba ambos e explique por que pertencem à mesma classe.

Nível 2: regras — Crie método aplicarDesconto(double percentual), evitando descontos fora de 0 a 100. Para produção, use BigDecimal.

Nível 3: MER — Separe OBRA, EDICAO, AUTOR e EXEMPLAR. Defina PK, FK e cardinalidades. Justifique as decisões.

Nível 4: SQL — Insira 3 livros. Selecione apenas os publicados após 2020. Atualize um título por id.

Nível 5: JDBC — Implemente buscarPorId(int id) com PreparedStatement. Trate livro não encontrado.

Nível 6: Swing — Crie formulário com título/ISBN/ano; valide entradas e conecte ao DAO.

Desafio extra — Implemente tela com JTable para consultar livros. Pense na execução fora da EDT.

Teste rápido: confira sua compreensão

1. Qual é a diferença entre classe e objeto?

2. Por que EXEMPLAR não é sinônimo de LIVRO?

3. Uma OBRA tem vários AUTORES: qual é a cardinalidade?

4. O que faz PreparedStatement?

5. Para SELECT, qual método JDBC devolve ResultSet?

6. Qual componente Swing representa a janela?

Gabarito comentado e encerramento

1 → Classe é um molde; objeto é uma instância concreta.

2 → Livro/obra descreve o conteúdo; exemplar representa uma cópia física identificável.

3 → N:N, normalmente resolvido por entidade associativa OBRA_AUTOR.

4 → Permite SQL parametrizado e evita concatenar dados diretamente ao comando.

5 → executeQuery().

6 → JFrame.

Ordem recomendada para quem faltou

Primeiro: execute o celular e entenda new, atributos, método e printf. Segundo: desenhe no brModelo quatro entidades e cardinalidades. Terceiro: crie a tabela LIVRO no MySQL. Quarto: execute Java com JDBC. Quinto: coloque uma tela Swing na frente do DAO. Assim cada etapa faz sentido.

Critério de sucesso

Você terminou quando consegue explicar o que cada classe faz, desenhar uma relação 1:N, criar tabela com chave primária, executar INSERT e SELECT, interpretar ResultSet e dizer o caminho completo de um clique até o banco de dados.

NA PRÁTICA | Ler ajuda. Rodar, errar, corrigir e explicar para outra pessoa é o que consolida o aprendizado.